IoT Challenge 2

Exercise Report – Part 2

Marco Abbadessa - 10854816 - leader
Alice Berta - 10889590

Academic Year 2025–2026

Contents

1 Introduction **2**

2 Scenario Description **2**
2.1 Assumptions 2

3 EQ1 – Communication Energy over One Hour **3**
3.1 Case (a): CoAP (Direct Communication) 3
3.2 Case (b): MQTT (via the Gateway) 3

4 EQ2 – Protocol Choice with Asymmetric Wake-Up Patterns **4**
4.1 Problem Description 4
4.2 Why CoAP is Not Suitable 4
4.3 Chosen Solution: MQTT with `retain = 1` 5
4.4 Protocol Choice 5

5 Final Answers Summary **5**

6 Conclusion **6**

1 Introduction

This document contains the solution to Part 2 of IoT Challenge 2, which concerns the communication energy consumption of a smart building system under different protocol choices. The exercise considers two alternative communication models between a battery-powered temperature sensor and a battery-powered actuator: direct communication through CoAP and indirect communication through an MQTT gateway.

The objective is twofold. First, the exercise requires the computation of the total communication energy consumed in one hour under the two protocol options. Second, it asks which protocol should be preferred in a modified scenario in which the actuator wakes up much less frequently than the sensor.

2 Scenario Description

The considered smart building system includes three actors. A **battery-powered temperature sensor** transmits one measurement every 2 minutes. A **battery-powered actuator**, acting as a fan controller, receives the temperature values and decides whether to switch on or off. A **gateway** is mains-powered and only supports MQTT, acting as an intermediary whenever MQTT communication is used.

The sensor and the actuator can exchange data in two alternative ways: directly, through CoAP, or indirectly through the gateway, using MQTT.

The communication parameters are fixed throughout the exercise: the payload size is 12 bytes and the topic or resource name length is 8 bytes. The message sizes provided by the exercise, already including protocol overhead and payload, are summarized in Figure 1.

COAP		MQTT	
CON PUT Request	70 B	Subscribe	65 B
PUT ACK	15 B	Sub Ack	55 B
		Publish	75 B
		Pub Ack	50 B
		Connect/Disconnect	60 B
		Connect Ack	50 B

Figure 1: *Message sizes given in the exercise statement.*

2.1 Assumptions

The following assumptions apply throughout all calculations:

- Transmit energy cost:
$$E_{TX} = 50\,\text{nJ/bit}$$
- Receive energy cost:
$$E_{RX} = 58\,\text{nJ/bit}$$
- CoAP communication uses Confirmable (`CON`) messages.
- MQTT communication uses QoS 1 on both the sensor and actuator sides.
- The communication channel is ideal, therefore no packet losses occur and no retransmissions are triggered in the base calculations.

- If retransmissions were needed, the protocol parameters would be

$$\text{RTO} = 30\,\text{s}, \qquad \text{MAX_TRANSMISSIONS} = 3$$

- Devices start in a disconnected state.
- MQTT connections are closed gracefully during sleep periods through a non-acknowledged `DISCONNECT`.
- Only the communication energy consumed by the sensor and the actuator is considered. The gateway energy is neglected because it is mains-powered.

3 EQ1 – Communication Energy over One Hour

To compute the total communication energy consumed over one hour, we first define the number of transmission cycles. Since the sensor transmits a measurement every 2 minutes, there are exactly 30 transmission cycles in one hour.

Furthermore, based on the specific assumptions provided for this scenario, we assume that for the steady-state 1-hour calculation, both nodes establish their connections at the beginning of the hour and **remain connected** without going to sleep or disconnecting.

3.1 Case (a): CoAP (Direct Communication)

CoAP operates on a stateless request/response model. For each of the 30 cycles, the sensor sends a Confirmable (`CON`) PUT Request directly to the actuator, which replies with a PUT ACK.

Sensor Energy per Cycle:

- Transmits `CON` PUT Request (70 B): $70 \times 8 \times 50 = 28,000\,\text{nJ} = 28\,\mu\text{J}$
- Receives PUT ACK (15 B): $15 \times 8 \times 58 = 6,960\,\text{nJ} = 6.96\,\mu\text{J}$
- **Total Sensor:** $34.96\,\mu\text{J}$

Actuator Energy per Cycle:

- Receives `CON` PUT Request (70 B): $70 \times 8 \times 58 = 32,480\,\text{nJ} = 32.48\,\mu\text{J}$
- Transmits PUT ACK (15 B): $15 \times 8 \times 50 = 6,000\,\text{nJ} = 6\,\mu\text{J}$
- **Total Actuator:** $38.48\,\mu\text{J}$

Total CoAP Energy (1 Hour):

$$E_{\text{CoAP}} = 30 \times (34.96 + 38.48) = 30 \times 73.44 = \mathbf{2203.2}\,\mu\text{J}$$

3.2 Case (b): MQTT (via the Gateway)

In the MQTT scenario, the devices must perform an initial setup phase to connect to the broker and subscribe to the topic. Because they do not disconnect during this hour, this setup penalty is paid exactly once. Afterwards, the periodic data exchange occurs 30 times using QoS 1.

1. Initialization Phase (Happens Once):

- **Sensor:** Transmits Connect (60 B) and receives Connect Ack (50 B).

$$E_{\text{sens_init}} = (60 \times 8 \times 50) + (50 \times 8 \times 58) = 47,200\,\text{nJ} = 47.2\,\mu\text{J}$$

- **Actuator:** Transmits Connect (60 B) + Subscribe (65 B) and receives Connect Ack (50 B) + Sub Ack (55 B).

$$E_{\text{act_init}} = (125 \times 8 \times 50) + (105 \times 8 \times 58) = 98,720\,\text{nJ} = 98.72\,\mu\text{J}$$

2. Periodic Communication (Happens 30 Times):

- **Sensor:** Transmits Publish (75 B) and receives Pub Ack (50 B).

$$E_{\text{sens_cycle}} = (75 \times 8 \times 50) + (50 \times 8 \times 58) = 53,200\,\text{nJ} = 53.2\,\mu\text{J}$$

- **Actuator:** Receives Publish (75 B) and transmits Pub Ack (50 B).

$$E_{\text{act_cycle}} = (75 \times 8 \times 58) + (50 \times 8 \times 50) = 54,800\,\text{nJ} = 54.8\,\mu\text{J}$$

Total MQTT Energy (1 Hour):

$$E_{\text{MQTT_Sensor}} = 47.2 + (30 \times 53.2) = 1643.2\,\mu\text{J}$$
$$E_{\text{MQTT_Actuator}} = 98.72 + (30 \times 54.8) = 1742.72\,\mu\text{J}$$
$$E_{\text{MQTT_Total}} = 1643.2 + 1742.72 = \mathbf{3385.92}\,\mu\text{J}$$

4 EQ2 – Protocol Choice with Asymmetric Wake-Up Patterns

4.1 Problem Description

In the modified scenario, the actuator wakes up only once every 30 minutes, instead of every 2 minutes, while the sensor continues to transmit one measurement every 2 minutes. This creates a strong temporal asymmetry between the two battery-powered devices. Over one hour, the sensor performs

$$N_{\text{sensor}} = \frac{60}{2} = 30$$

transmission cycles, whereas the actuator wakes up only

$$N_{\text{act}} = \frac{60}{30} = 2$$

times.

As a consequence, the communication protocol should be chosen not only on the basis of correctness, but also on its ability to cope efficiently with a producer that is active much more often than the consumer.

4.2 Why CoAP is Not Suitable

With direct CoAP communication based on Confirmable PUT requests, the sensor sends a request and expects an ACK from the actuator. If the actuator is sleeping, no ACK is returned. According to the assumptions of the exercise, the sensor would then attempt retransmissions, consuming additional energy for transmissions that cannot succeed while the actuator remains asleep. After the maximum number of attempts is reached, the measurement is lost.

One could consider an alternative CoAP-based design using the Observe mechanism, where the actuator registers as an observer of a sensor resource and the sensor pushes notifications whenever new values are available. However, this still does not solve the core problem. Notifications would continue to be sent while the actuator is sleeping, missing acknowledgements would eventually break the observe relation, and the actuator would need to re-establish the subscription after waking up.

In other words, CoAP does not provide a convenient broker-side buffering mechanism for this strongly asynchronous communication pattern. For this reason, it is not the most suitable choice for long-term operation in the modified scenario.

4.3 Chosen Solution: MQTT with `retain = 1`

The recommended solution is **MQTT with QoS 1 and `retain = 1`** on the sensor publishes. In addition, the actuator should use a **persistent session**, i.e. `clean_session = 0`.

This choice is motivated by the broker-based nature of MQTT. When the sensor publishes a retained message, the broker stores the most recent value for that topic. Therefore, even if the actuator remains asleep for a long time, the latest temperature value is preserved at the broker. When the actuator wakes up and reconnects, it can immediately receive the most recent retained message without requiring the sensor to retransmit it at that exact moment.

A persistent session further improves the solution. After the first subscription, the broker remembers the actuator's subscription state across disconnections. Consequently, when the actuator reconnects, it does not need to subscribe again at every wake-up, which reduces communication overhead and saves additional energy on the actuator side.

The resulting behavior is the following. The sensor continues to wake up every 2 minutes, connects to the broker, publishes the temperature with QoS 1 and `retain = 1`, and disconnects. The broker continuously updates the last stored value for that topic. The actuator wakes up only every 30 minutes, reconnects with persistent session enabled, receives the retained value, acknowledges it with a PUBACK, performs the control decision, and goes back to sleep.

4.4 Protocol Choice

From a qualitative point of view, MQTT is preferable because it decouples the timing of the producer and the consumer. The sensor can continue to publish at its own pace, while the actuator can sleep for long intervals without causing failed transmissions or repeated retries. The broker acts as temporary storage for the latest relevant information.

From an energy point of view, this is especially advantageous for the actuator, which is the device affected by the sparse wake-up pattern. Its communication overhead is limited to only two short wake-up exchanges per hour while CoAP would suffer from repeated failed attempts or from the need to continuously rebuild state after each wake-up.

For these reasons, MQTT with retained messages and persistent session is the best choice for long-term operation in this scenario.

EQ2 answer: We would choose MQTT with QoS 1, `retain = 1`, and persistent session (`clean_session = 0`) for the actuator. The broker stores the latest temperature value while the actuator sleeps and delivers it as soon as the actuator reconnects. This avoids wasted retransmissions and minimizes actuator-side communication energy.

5 Final Answers Summary

- **EQ1:** The total communication energy consumed over one hour is **2203.2** µJ using CoAP and **3385.92** µJ using MQTT.
- **EQ2:** MQTT with QoS 1, `retain = 1`, and persistent session (`clean_session = 0`).

6 Conclusion

This exercise highlights the importance of evaluating both raw communication overhead and systemic behavior when selecting IoT protocols. For a symmetric, high-frequency data exchange where devices remain continually active (EQ1), CoAP is slightly more energy-efficient (2203.2 µJ vs 3385.92 µJ) due to the lack of connection and subscription overhead.

However, as demonstrated in EQ2, real-world IoT deployments frequently involve asynchronous wake-up patterns to conserve battery life. In these scenarios, the stateless, synchronous nature of CoAP breaks down, leading to wasted energy via retransmissions. MQTT, acting as a stateful broker, elegantly bridges this temporal gap. By leveraging retained messages and persistent sessions, MQTT allows the sensor to publish at its own pace while letting the actuator sleep deeply, ultimately proving to be the superior and more resilient choice for long-term, battery-powered operation.